

Downlink-Constrained Task Scheduling for Orbital Edge Computing: Grounding Utility in Measured Downstream Task Performance

Hemanth Sudhakaran
NICE Lab, North Carolina State University

September 10, 2026

Abstract

Low-Earth-orbit (LEO) satellites capture far more imagery than their downlink budget can deliver in time to be useful, forcing an on-board choice of *what* to send, *how much* to compress it, and *when*. This paper studies that choice for Orbital Edge Computing (OEC): a residual-quantized VAE (RQ-VAE) compresses each image at one of several selectable depths, and a rolling-horizon Model Predictive Control (MPC) scheduler decides depth, timing, and routing under time-varying satellite/ground-station connectivity. Our central finding is that the choice of *quality signal* used to reward a scheduler determines whether MPC’s benefit is even visible: under a pixel-fidelity proxy (LPIPS), MPC beats the best fixed-depth baseline by only 1.2%; once quality is grounded in the downstream task the imagery actually serves (measured semantic-segmentation mIoU), the same comparison shows a 32.2% margin — roughly $26\times$ larger — because pixel fidelity is nearly flat across compression depths while task-relevant quality is not (+12% vs. +247% across the same depth range). Building on this, we present a single unified utility function reconciling five previously disagreeing scoring definitions, evaluate ten schedulers (fixed-depth and adaptive greedy baselines, flat MPC, congestion-aware routing, and two hierarchical/two-level MPC couplings) against an offline optimality bound, and analyze the resulting depth-selection decisions and sensitivity to link-rate scarcity. We find that admission control — not routing sophistication — is what separates the strongest and weakest MPC variants, and that freezing a concrete route for longer than a few minutes does not survive a LEO constellation’s short ground-contact windows. A reinforcement-learning (PPO) baseline is also evaluated: it trades a consistent 5–8% utility gap for a $250\text{--}400\times$ reduction in per-decision compute time.

1 Introduction

A satellite in low Earth orbit (LEO) can observe far more of the ground than it can ever report. A single imaging pass over a wildfire, a flood, or an agricultural region can generate far more raw pixels than the satellite’s downlink budget can move to a ground station before the next useful decision has to be made — and a LEO satellite is only in radio contact with any one ground station for a few minutes at a time before it drops below the horizon. Someone, or something, has to decide which images matter most, how aggressively to compress each one, and when to send it. *Orbital Edge Computing* (OEC) is the idea of making some of that decision on board the satellite itself — compressing and prioritizing at the edge of the network, rather than shipping raw data down and sorting it out on the ground.

This paper studies that decision problem end to end: an on-board residual-quantized variational autoencoder (RQ-VAE) compresses each image at one of several selectable *depths* (more refinement passes, larger payload, higher fidelity), and a rolling-horizon Model Predictive Control (MPC)

scheduler decides, for every task competing for downlink capacity, which depth to use and when and along which route to transmit it, under a constellation topology and ground-station contact schedule that changes every scheduling slot.

The problem with grading a scheduler on how the image looks. The natural way to score a compression-and-scheduling policy is to ask whether the reconstructed image still *looks* like the original — standard fidelity metrics such as PSNR and LPIPS answer exactly that question. But for a mission whose actual goal is to extract information from the imagery (what land-cover class is this, where is the flood boundary), *looking similar* and *remaining useful for the task* are not the same property, and this paper’s first and most consequential finding is that they diverge sharply here: pixel-fidelity quality changes by only 12% across the compression-depth range we test, while the imagery’s usefulness for a real downstream semantic-segmentation task changes by 247% over the identical range (§5.3). A scheduler rewarded on the flat proxy has almost nothing to gain by reasoning carefully about depth; a scheduler rewarded on the true task-relevant signal has a great deal to gain. This single substitution — replacing a pixel-fidelity proxy with a measured downstream-task quality signal — changes MPC’s measured advantage over the best fixed-depth baseline from a marginal 1.2% to a substantial 32.2% in an otherwise identical experiment (§6).

Contributions. Concretely, this paper:

1. grounds a scheduling utility function’s quality term in *measured* downstream semantic-segmentation mIoU rather than pixel fidelity, and shows this changes not just the numbers but the qualitative conclusion about whether intelligent scheduling matters (§5, §6);
2. presents a single *unified utility function* — quality, tardiness, resource cost, and a fairness floor, combined so that the resulting optimization stays a mixed-integer *linear* program with no auxiliary binary variables — that replaces five previously disagreeing scoring definitions used across the scheduler, the offline bound, and two prior hierarchical variants (§3);
3. evaluates ten schedulers — four fixed-depth and one adaptive greedy baseline, a flat rolling-horizon MPC, a congestion-aware routing variant, and two ways of coupling a dedicated routing optimizer to the depth decision (hierarchical and two-level/peer) — against a three-tiered offline optimality bound, under two link-capacity regimes designed to make different resources binding (§4, §6);
4. shows, from the resulting depth-selection decisions and a link-rate sensitivity sweep, that *admission control* — deciding early to drop a task rather than let it starve — is what separates the strongest and weakest MPC variants, not how cleverly the kept tasks are routed (§7, §8); and
5. reports a structural, not tunable, negative result: freezing a concrete route for a multi-minute planning epoch is usable at its own executed slot only a small fraction of the time, because ground-station contact windows in this constellation are shorter than any freeze period worth having (§3, §6); and we additionally evaluate a learned (PPO) scheduling policy as a compute-cost point of comparison to MPC (§9).

Roadmap. §2 situates this work relative to orbital edge computing, learned image compression, and MPC-based network scheduling. §3 gives the system model and the full scheduling formulation, including every MPC variant evaluated later. §4 specifies the simulated constellation, link-capacity regimes, and task workload used to reproduce every subsequent result. §5–§9 present results: compression and downstream-task quality, the unified-utility scheduler comparison, depth-selection decision distributions, link-rate sensitivity, and the learned-policy baseline, in that order. §10 summarizes and discusses what remains open.

2 Related Work

[CONFIRM WITH XUANHAO: This section is drafted against general background rather than the two reference papers Xuanhao sent for how experimental results should be presented — those weren’t available in the session that drafted this paper. Once shared, this section’s structure/tone and citation set should be revised to match them; the citations below are limited to works this project directly builds on, to avoid citing anything not actually verified.]

Orbital edge computing and LEO satellite networking. Moving computation to the network edge is a well-established idea in terrestrial systems; OEC applies the same principle where the “edge” is the satellite itself, motivated by a downlink budget that is orders of magnitude smaller than the volume of imagery a modern sensor can capture in a single pass. LEO constellations add a networking wrinkle largely absent from terrestrial edge computing: topology and link capacity are not merely variable but *predictable* — satellite and ground-station positions, and therefore contact windows and candidate routes, are known in advance from orbital mechanics. This paper’s simulation environment is built on Hypatia [1], an open-source LEO network simulator that models constellation geometry, inter-satellite links, and ground-station contact windows explicitly, which is what makes a rolling-horizon scheduler that *predicts* short-term future connectivity (rather than only reacting to it) both possible and testable at scale.

Learned image compression for downlink-constrained systems. On-board compression here uses a residual-quantized variational autoencoder (RQ-VAE) [2]: an encoder maps an image to a small set of codebook indices, and successive *residual* quantization stages progressively refine the reconstruction, each stage correcting the error left by the stage before it. The number of stages run before transmission — this paper’s *depth* — is exactly the fidelity/payload knob a scheduler has to reason about. We evaluate this compression model on two remote-sensing benchmarks: EuroSAT [3], a 10-class land-cover classification dataset used for an initial spatial-size/depth sweep, and FLAIR-1 [4], a larger aerial-imagery semantic-segmentation dataset (15 land-cover classes) used for the paper’s central downstream-task evaluation. Unlike most learned-compression evaluations, which report pixel-fidelity metrics (PSNR, SSIM, LPIPS, FID) as an end in themselves, this paper treats those metrics as necessary but insufficient: our results (§5.3) show they can be nearly flat exactly where a downstream task’s performance is not.

MPC-based scheduling and network optimization. Model Predictive Control — solve an optimization over a finite look-ahead horizon, act on only the first step, then re-plan as new information arrives — is a natural fit for a system whose future state (contact windows, link capacities) is partially predictable but subject to real uncertainty (bursty task arrival, imperfect topology forecasts). We formulate the scheduling problem as a mixed-integer linear program so it remains tractable at the problem sizes this paper evaluates, and construct a piecewise-linear coverage reward specifically so that a concave, diminishing-returns objective can be optimized without introducing additional binary or SOS2 variables (§3.4) — a standard technique for keeping a naturally nonlinear objective inside a linear program’s reach. We evaluate this rolling-horizon approach against both simpler greedy baselines and a learned (reinforcement-learning) policy (§9), and against hierarchical and peer couplings that split routing and depth decisions into separate, communicating optimizers (§3).

3 System Model and Problem Formulation

This section states the system model and scheduling formulation implemented and evaluated in this paper. It builds directly on the original system-and-task model and rolling-horizon MPC objective (satellites $\mathcal{S}$, GBSs $\mathcal{G}$, tasks $\mathcal{K}$, depths $\mathcal{D}$, and the basic MPC structure), extending it with the concrete quality signal, unified multi-term utility, and additional scheduler variants (congestion-aware routing, two-level and hierarchical MPC couplings, and an offline optimality bound) evaluated in

§4 onward. Every symbol below matches the implementation (`hypatia_sim/oec_sim/`) exactly; any place where an earlier design intent and the shipped code differ is flagged inline rather than silently resolved.

3.1 Sets, time, and links

We consider a LEO orbital edge computing network consisting of a set of satellites $\mathcal{S}$ and a set of ground base stations (GBSs) $\mathcal{G}$. Time is divided into equal scheduling slots $t \in \mathcal{T}$, each of duration Δt . Because satellites move, the network topology is time-varying: let $\mathcal{E}_t$ denote the set of available inter-satellite links (ISLs) and satellite-to-ground links at slot t , where each available link $(i, j) \in \mathcal{E}_t$ has capacity $C_{ij}(t)$, obtainable from predicted contact windows and link states. In this paper, $\mathcal{S}$ is a 1,156-satellite Kuiper-630-shell Walker constellation, $\mathcal{G}$ is a 4-city ground segment, $\Delta t = 30$ s, and $\mathcal{E}_t$ is gated by a line-of-sight/range test for ISLs and an elevation test for ground links — full numeric values are in Table 1 (§4).

3.2 Tasks and compression depths

A set of image-transmission tasks $\mathcal{K}$ is generated when satellites observe fixed areas of interest. Each task $k \in \mathcal{K}$ is characterized by its source satellite s_k , destination GBS g_k , arrival time a_k , number of images N_k , importance weight w_k , and soft deadline d_k . Each task is compressed using one effective RQ-VAE depth selected from

$$\mathcal{D} = \{2, 4, 8, 16\}. \quad (1)$$

In plain terms: every image in a task is encoded at exactly one of four possible compression strengths, and that single choice is what trades off payload size against how useful the image ends up being once reconstructed.

For each depth $q \in \mathcal{D}$, let b_q denote the compressed payload size per image and t_q^{enc} the encoding delay. Rather than leaving downstream utility as an abstract placeholder u_q , this paper *measures* it: $b_q = 88q$ bytes (measured on the FLAIR-1 8×8 dedicated models, §5), and quality is either the legacy pixel-fidelity proxy $u_q = 1 - \text{LPIPS}_q$ or the measured downstream segmentation quality s_q (both selectable via a configuration flag, see §5.3 for why the latter is used throughout the rest of this paper):

$$s_q = \frac{\text{mIoU}_q - \text{mIoU}_{\text{floor}}}{\text{mIoU}_{\text{ref}} - \text{mIoU}_{\text{floor}}}, \quad (2)$$

where $\text{mIoU}_{\text{floor}}$ is the downstream segmentation model’s score with the optical bands blanked out (near-infrared and elevation channels only). *In plain terms:* this rescales raw mIoU so that $s_q = 0$ means ”as useless as not delivering the image’s optical content at all” (the decision-theoretic floor) rather than the mathematical floor $\text{mIoU} = 0$, and $s_q = 1$ means ”as useful as the uncompressed original” — so s_q is directly interpretable as a fraction of the achievable task value recovered at depth q .

Let $x_{k,q} \in \{0, 1\}$ indicate whether task k is compressed using depth q . Each task selects exactly one compression depth:

$$\sum_{q \in \mathcal{D}} x_{k,q} = 1, \quad \forall k \in \mathcal{K}. \quad (3)$$

Accordingly, the total compressed payload size of task k is

$$B_k = N_k \sum_{q \in \mathcal{D}} b_q x_{k,q}. \quad (4)$$

In plain terms: these two equations just say ”pick one depth per task” and ”the task’s total size is its image count times whichever per-image payload that depth costs” — they are the accounting that every later constraint and objective term builds on.

Let $r_k(t)$ denote the number of images of task k fully delivered to its destination GBS by the end of slot t . The delivery fraction of task k is

$$\rho_k(t) = \frac{r_k(t)}{N_k}, \quad 0 \leq \rho_k(t) \leq 1. \quad (5)$$

Let $y_{k,ij}(t) \geq 0$ denote the number of bits of task k transmitted over link (i, j) during slot t . The link-capacity constraint is

$$\sum_{k \in \mathcal{K}} y_{k,ij}(t) \leq C_{ij}(t) \Delta t, \quad \forall (i, j) \in \mathcal{E}_t. \quad (6)$$

In plain terms: no link can be asked to carry more traffic in one slot than its capacity allows — every task sharing a link must fit within that link’s slot budget together.

Let $Q_{i,k}(t)$ denote the remaining amount of task k ’s data stored at node i at slot t . The queue state evolves according to arriving compressed data and transmitted traffic, subject to standard flow conservation over the time-varying network: data can only leave a queue by being transmitted along a feasible link, and cannot exceed what has actually arrived at that node so far.

To capture latency sensitivity, let $\phi_k(t) \in [0, 1]$ denote the freshness weight of task k at slot t :

$$\phi_k(t) = \begin{cases} 1, & t\Delta t \leq d_k, \\ \exp(-\alpha_k(t\Delta t - d_k)), & t\Delta t > d_k, \end{cases} \quad (7)$$

where α_k controls how quickly value decays after the deadline ($\alpha = 1/300 \text{ s}^{-1}$ in this paper’s experiments). *In plain terms:* an image delivered before its soft deadline counts at full value; one delivered late is still worth something, but that value decays exponentially the later it arrives — a wildfire photo that shows up a day late is close to worthless, one that is five minutes late is nearly as good as on time. Tasks $|\mathcal{K}| = 64$ arrive from 8 areas of interest with $N_k \sim \mathcal{U}\{100,000, 300,000\}$ and $w_k \sim \mathcal{U}\{1, 2, 3\}$ (seed 42); a task may be explicitly **dropped** (abandoned mid-flight) or **rejected** (refused at admission), both first-class reported outcomes rather than silent non-delivery.

3.3 Unified utility

Utility used to be defined in five places that disagreed: the reported score, the flat MPC’s own objective, the two levels of the hierarchical scheduler, and the offline bound each used a different combination of terms. This paper instead uses one function for all five. With normalized weight $\bar{w}_k = w_k / \max_j w_j$ and realized arrival time $t^{\text{arr}} = (t+1)\Delta t + \delta_{k,t}$ along the path actually used for delivery,

$$U_k = \bar{w}_k \left[\omega_Q s_{q_k} \hat{G}_k - \omega_T T_k - \omega_E C_k \right], \quad (8)$$

$$U = \sum_k U_k + \omega_F |\mathcal{K}| \min_k \hat{U}_k, \quad (9)$$

with default weights $\omega_Q = 1.0$, $\omega_T = 0.25$, $\omega_E = 0.05$, $\omega_F = 0.1$. *In plain terms:* each task earns a reward proportional to how much of it was delivered, scaled by how good the compression quality was (s_{q_k}) and how much of it actually got through ($\hat{G}_k$); that reward is then reduced by a lateness penalty (T_k) and a resource-cost penalty (C_k); and the whole system’s score adds up every task’s net reward, plus a bonus tied to whichever single task did *worst* (the fairness floor), so a scheduler cannot rack up a high score by quietly abandoning one inconvenient task. Here $\hat{G}_k \in [0, 1]$ is a concave coverage gain over delivered, freshness-weighted image fraction (§3.4); tardiness is

$$T_k = \sum_t \frac{\Delta r_{k,t}}{N_k} \min\left(1, \frac{\max(0, t^{\text{arr}} - d_k)}{\Delta_{\text{ref}}}\right), \quad (10)$$

i.e. each newly-delivered slice of images contributes a penalty proportional to how late it arrived past the deadline, capped at 1 (a slice cannot be penalized as ”more than maximally late”); resource

cost is

$$C_k = \sum_t \frac{\Delta r_{k,t}}{N_k} \left[\hat{\omega}_{\text{tx}} \frac{b_q}{b_{\text{max}}} + \hat{\omega}_{\text{enc}} \right], \quad (11)$$

a normalized charge for the bandwidth and encode effort spent on each delivered slice; and $\hat{U}_k = U_k / (\omega_Q s_{\text{max}} \bar{w}_k) \leq 1$ is a scale-free, weight-relative normalization of U_k used only inside the fairness term, so that a naturally "small" task (low weight) is not unfairly flagged as the worst-off one just because its raw U_k is small.

Legacy identity. Setting $\omega_Q=1$, $\omega_T=\omega_E=\omega_F=0$, one coverage segment of unit width/slope, no weight normalization, and $s_q = 1 - \text{LPIPS}_q$ collapses this exactly to the original score $\sum_k \sum_t w_k \phi_k(t^{\text{arr}}) u_{q_k} \Delta r_{k,t} / N_k$ — the legacy scoring is a parameter setting of this same function, not a separate code path, and every previously-committed legacy number reproduces exactly under this setting.

3.4 Concave coverage without integer variables

Getting the first fraction of a task's images delivered is worth more, per image, than getting the last fraction — a *concave*, diminishing-returns reward. Normally encoding a curved reward inside a linear program requires extra machinery (auxiliary binary or SOS2 variables, which are expensive to solve). Instead, coverage gain is linearized over $J=4$ segments of width Δ_j and strictly decreasing slopes $m_1 > \dots > m_J$ with $\sum_j \Delta_j m_j = 1$ (default: widths (0.25, 0.25, 0.25, 0.25), slopes (2.00, 1.12, 0.60, 0.28), a discretization of $g(\rho) = (1 - e^{-3\rho}) / (1 - e^{-3})$):

$$\hat{G}_k = \sum_{\tau,j} m_j \phi_k(t_{\tau}^{\text{arr}}) \lambda_{k,q,\tau,j}, \quad (12)$$

$$\sum_j \lambda_{k,q,\tau,j} = \frac{y_{k,q,\tau}}{N_k}, \quad \sum_{q,\tau} \lambda_{k,q,\tau,j} \leq \Delta_j^{\text{res}}, \quad (13)$$

where Δ_j^{res} is the width of segment j still unclaimed by coverage already realized before the current planning horizon (omitting this residual would let the MPC re-earn steep early-segment credit at every re-plan, double-counting value already banked). *In plain terms:* think of each task's 0–100% delivered range as four stacked buckets, each paying less per percentage point than the one before it; $\lambda_{k,q,\tau,j}$ is how much of bucket j a given slot's delivery fills. Because the payoff always shrinks bucket by bucket, a solver — even though it only understands straight lines — will always fill bucket 1 before touching bucket 2, and so on, entirely on its own: no binary "which bucket am I in" variable is ever needed, and the linear relaxation of this objective is exact.

3.5 Fairness: maximin, not Jain

Jain's index $(\sum_k U_k)^2 / (|\mathcal{K}| \sum_k U_k^2)$ is a classic networking fairness statistic, but it is a ratio of quadratics and cannot be represented inside a mixed-integer *linear* program, so it is reported only as a diagnostic, never optimized directly. What is actually optimized is a *maximin* floor — literally, how well the single worst-off task did — via one continuous column $u_{\min}$ and $|\mathcal{K}|$ constraint rows $u_{\min} \leq \hat{U}_k$, entering the objective as $\omega_F |\mathcal{K}| u_{\min}$ (the second term of U above). $u_{\min}$ requires a free, unbounded-below lower bound, since $\hat{U}_k$ can go negative once tardiness/cost penalties exceed the quality gain for a badly-served task.

3.6 Flat MPC objective

At each decision slot t , the scheduler observes the current network state — task queues, remaining deadlines, satellite positions, available links — and predicts network evolution over a finite horizon of H slots: $\hat{\mathcal{E}}_{\tau|t}$, $\hat{C}_{ij}(\tau|t)$, $\hat{Q}_{i,k}(\tau|t)$ for $\tau = t, \dots, t+H-1$. It jointly determines compression depth, transmission timing, and traffic allocation over that horizon, then executes only the first-slot decisions; at slot $t+1$ it observes the new state, shifts the horizon forward, and solves again.

The rolling-horizon MILP solved by the flat `mpc` scheduler (re-solved every 5 slots or on new-task arrival; $H = 60$ slots) is

$$\max \sum_{k,q,\tau} \left[w_k \phi_k(t_{k,\tau}^{\text{arr}}) u_q + \lambda_1(H - \tau) b_q \cdot 8/10^9 - \lambda_{\text{late}} w_k \frac{\max(0, t_{k,\tau}^{\text{arr}} - d_k)}{\Delta_{\text{ref}}} \right] \frac{y_{k,q,\tau}}{N_k}, \quad (14)$$

with $\lambda_{\text{late}} = 5 \times 10^{-2}$, $\Delta_{\text{ref}} = 300$ s, subject to: one depth per task per slot, link/GBS capacity, the encoder throughput pipeline (81 images/s per satellite), and cumulative delivered-volume consistency. Here $t_{k,\tau}^{\text{arr}} = (t + \tau + 1)\Delta t + \text{route delay}$ uses the real propagation delay of the path actually assigned to (k, τ) . *In plain terms:* the objective rewards weighted, freshness-discounted useful delivery (first bracket term), gives a small forward-looking bonus to plans that keep transmitting rather than idling capacity (second term), and subtracts an explicit lateness penalty (third term) — letting the MPC *rationaly choose* to stop pushing a task’s backlog once its cost outweighs its already-decayed value, rather than that decision happening implicitly through silent non-delivery. §3.9 below makes this same tradeoff explicit via admission control and drop.

3.7 Congestion-aware predictive routing

The default router picks the geometrically shortest path regardless of how busy it currently is, which can overload a popular link while a nearly-as-fast alternative sits idle. The congestion-aware variant (`mpc-congestion`) instead predicts each link’s cost as propagation delay plus a queueing-theory-style congestion penalty computed against the MPC’s own current plan:

$$w_e(\tau) = d_e(\tau) + \beta d_{\text{ref}} \frac{\rho_e(\tau)}{1 - \min(\rho_e(\tau), \rho_{\text{max}})}, \quad \rho_e(\tau) = \frac{f_e(\tau)}{C_e \Delta t}, \quad (15)$$

where $f_e(\tau)$ is the bits the current plan places on edge e at predicted step τ , and $\rho_e(\tau)$ is that edge’s predicted utilization. *In plain terms:* a link’s effective cost stays close to its physical delay while mostly empty, then rises sharply as it approaches saturation ($\rho_e \rightarrow \rho_{\text{max}}$), which naturally spreads traffic across several good paths instead of jamming everyone onto the one “best on paper” route. Because the plan depends on routes and routes depend on the plan, this is iterated to a damped fixed point (method-of-successive-averages, $\eta_{\text{it}} = 1/(\text{it} + 2)$, 3 rounds, keep-best-iterate), with ρ clipped at 0.99 so a saturated link is never treated as disconnected. This coupling is only measurable under the fabric-limited capacity regime (§4) — under the gbs-limited regime the ground-station aggregate budget dominates every route regardless of ISL cost, and congestion-aware routing is a provable, verified no-op there.

3.8 Two-MPC couplings: routing and depth

`mpc-2level`: a dedicated routing MPC solves a min-cost multicommodity-flow linear program over candidate paths,

$$\min \sum_{k,p,\tau} f_{k,p,\tau} \frac{\delta_{p,\tau}}{\Delta_{\text{ref}}^{\text{delay}}} + M \sum_{k,\tau} \text{sh}_{k,\tau} \quad \text{s.t.} \quad \sum_p f_{k,p,\tau} + \text{sh}_{k,\tau} = D_{k,\tau}, \quad \sum_k \sum_{p \ni e} f_{k,p,\tau} \leq C_e \Delta t, \quad (16)$$

where $f_{k,p,\tau}$ is the flow assigned to task k on candidate path p , $\text{sh}_{k,\tau}$ is an explicit “shortfall” variable absorbing any demand no path can carry (penalized heavily by M so it is only used when genuinely unavoidable), and $D_{k,\tau}$ is task k ’s desired bit demand at step τ . *In plain terms:* this LP spreads each task’s traffic across whichever candidate paths have room, at minimum total delay, rather than committing everything to one path — and if no combination of paths can carry all of it, the shortfall variable makes that gap explicit instead of the LP becoming infeasible. This routing LP and the depth MILP (§3.6) exchange demand $D_{k,\tau}$ and per-edge flow shares, iterated to a fixed point with the same successive-averages damping as §3.7. Critically, $D_{k,\tau}$ must be the task’s *desired* bits, not what the depth MILP already conceded — deriving it from the previous plan would be circular and silently reproduce flat MPC even under real congestion (a correctness

trap caught during development). Iteration 0 is defined to be exactly flat `mpc`, so `mpc-2level` can never plan worse than flat MPC plans, by construction.

mpc-hier-route: a route-coordinator MPC runs once per 10-minute macro-epoch over a 60-slot horizon in 10-slot macro-windows (topology evaluated at each window’s middle slot, as a forecast), solving the same flow LP to *freeze* the top- K candidate paths per task for the whole epoch; the fast depth MILP then solves only within those frozen paths. Empty route assignments reproduce `mpc-hier` (§3.9) exactly, verified bit-for-bit.

Measured: route freezing does not survive a LEO fabric. A route frozen at epoch start is usable at its own executed slot only 14.4% of the time in general (this paper’s fabric-limited scenario: 7.7%), falling to 0% by a 570-second lookahead — because ground-station contact windows average only 227–265 s, shorter than any planning epoch worth having. This is a structural property of the constellation geometry, not a tuning failure: it is specifically a negative result about freezing *concrete paths*, not necessarily about freezing a more abstract routing decision (e.g. a serving-GBS assignment), which remains untested.

3.9 Hierarchical MPC and the rotting-backlog fix

`mpc-hier` pairs a slow upper-level coordinator (cadence 20 slots / 10 min, horizon 6 macro-steps $\approx$ 2 h, an LP over *continuous relaxed* depth $x_{k,q} \in [0, 1]$ and GBS-aggregate budget only) with a fast lower-level per-task MILP (cadence 5 slots or on arrival, horizon 20 slots, *integral* depth $x_{k,q}$, the same machinery as §3.6). *In plain terms:* the upper level decides, at a coarse cadence, which tasks get admitted and roughly how much budget each gets; the lower level decides, at a fine cadence, the exact depth and schedule within that budget — trading a smaller, faster-solved problem for real admission/drop activity. Three mechanisms address a specific failure mode, *stale-backlog starvation* (a task whose freshness has decayed to near zero but still generates optimization variables and is never served nor removed): admission control at arrival, explicit drop once freshness falls below a threshold or a task is sufficiently overdue (reported as **dropped**, not hidden), and a backpressure aging term in the lower-level objective that raises priority the longer a task has waited. These thresholds are first-pass tuned (five combinations tried), not an exhaustive search.

mpc-hier vs. the routing/depth split. `mpc-hier`’s two levels both operate on *depth* (a relaxed depth budget above, an integral depth choice below), with static routing at both levels — a different pairing from `mpc-2level/mpc-hier-route`, whose split is one MPC dedicated to *routing* and one to *depth*. [CONFIRM WITH XUANHAO: Xuanhao’s email asked to drop “the previous mpc-hier results where the two MPCs are both used for depth selection.” That description textually matches `mpc-hier` as just defined, which makes it the likely candidate for what Xuanhao means — but all scheduler variants are still reported in §6 pending his direct confirmation of scope.]

3.10 Offline optimality bound

To know whether a scheduler’s score (e.g. 18.9) is actually good, it must be compared to the best *possible* score under the same rules — an offline bound, computed here three ways of increasing tightness: (1) an analytic ceiling (every image at the best depth, delivered instantly — a sanity check only, not a meaningful bound); (2) a linear-programming bound (depth relaxed to continuous $x_{k,q} \in [0, 1]$, time aggregated into 5-slot windows, provably-non-binding link/GBS rows dropped, utility evaluated at each window’s *start* to remain a genuine upper bound — never optimistic about when delivery actually happens); and (3) a mixed-integer bound (the same relaxed constraints at full time resolution with integral depth, reporting the solver’s dual bound even when the search has not fully converged within its time budget, which remains a valid upper bound regardless). The governing correctness rule is that every term in a scheduler’s realized score must also appear in the bound’s objective, relaxed only in the optimistic direction — enforced at runtime by an assertion that the bound is never smaller than any scheduler’s realized score.

4 Experimental Setup

This section specifies every parameter needed to reproduce the results in this paper from the same simulator (`hypatia_sim/oec_sim/`, built on the Hypatia LEO orbital-mechanics/topology engine).

4.1 Constellation and ground segment

Table 1 lists the physical scenario parameters (`hypatia_sim/oec_sim/config.py`). Four named constellations are implemented (`kuiper-630`, `starlink-550`, `telesat-1015`, `small-walker`); all results in this paper use `kuiper-630` unless stated otherwise.

Table 1: Constellation, link, and task-load parameters.

Parameter	Value
Constellation	Kuiper-630 shell 1: Walker-delta 1156/34/1 (34 planes $\times$ 34 sats)
Altitude / inclination	630 km / 51.9°
Orbital period	5,830 s
Ground stations G	Tokyo, New York, São Paulo, Sydney (4 total), min. elevation 20°
ISL topology	+Grid, 2,312 candidate links; feasible if graze altitude > 80 km above Earth’s surface <i>and</i> range $\leq 5,016$ km
Areas of interest	8 (California-fires, Amazon-basin, Sahel, Ganges-delta, Barrier-reef, Dnipro-basin, and 2 more; see <code>config.py:A0IS</code>)
Tasks per run $ K $	64, one per (AOI, publication event), Poisson-triggered
Images per task N_k	$\sim \mathcal{U}(100,000, 300,000)$, total ≈ 12.81 M images
Task weight w_k	$\in \{1, 2, 3\}$
Random seed	42 (task generation; sweeps in §8 use 5 seeds)
Simulation horizon	18,000 s (5 h, ≈ 3.1 orbits)
Slot length Δt	30 s $\Rightarrow$ 601 decision slots
MPC planning horizon H	60 slots (30 min look-ahead)

4.2 Link capacity regimes

Two named capacity regimes share the identical topology and task model but rebalance link rates to make different resources binding (Table 2). **gbs-limited** is the default and is used for the legacy-vs-unified utility comparison in §6; its per-GBS aggregate budget is $\geq 12\times$ tighter than any ISL segment a route could cross, so ISL contention is mathematically unreachable there. **fabric-limited** throttles the ISL rate specifically so routing decisions have a measurable effect, and is used for the ten-scheduler comparison in §6 and the decision-distribution analysis in §7.

Table 2: Link capacity regimes (`config.py:_SCENARIOS`).

Regime	ISL rate	GSL rate (per sat-GBS pair)	GBS aggregate rate
gbs-limited (default)	100 Mbps	100 Mbps	2 Mbps
fabric-limited	1 Mbps	50 Mbps	500 Mbps

4.3 Compression depths and payload

Each task-image is encoded on board at one of $D = \{2, 4, 8, 16\}$ RQ-VAE residual-quantization depths before downlink. Payload scales linearly with depth, $b_q = 88q$ bytes (measured on the FLAIR-1 8×8 dedicated models, §5): $b_2 = 176$ B, $b_4 = 352$ B, $b_8 = 704$ B, $b_{16} = 1,408$ B. On-board encode time is 12.34 ms $\pm$ 0.03–0.05 ms per image and is *depth-independent* (measured identically

at $8 \times 8 \times 1$ and $8 \times 8 \times 8$), giving an encoder throughput of 81 images/s per satellite regardless of the depth chosen.

4.4 Decision cadence: what “every 30 s” actually means

The scheduling slot is $\Delta t = 30$ s, but the MPC does *not* re-solve its optimization every slot. Each solve plans $H = 60$ slots (30 min) ahead and commits only to the first slot of that plan; the scheduler re-solves only when either 5 slots have elapsed since the last solve, or a new task arrives — otherwise it executes the next committed step of the existing plan. Over the full 601-slot (5-hour) simulation, the flat mpc scheduler solves the optimization approximately 121 times, not 601 times. **Reported solve-time statistics (mean/p95/max) are for a single individual solve** (how long the optimizer takes to think once); a separately reported *total* solve-time figure is the sum of individual solve times across the whole run — total compute time spent solving, not wall-clock simulation time and not a per-slot cost.

4.5 Software and solver

The scheduling MILPs/LPs (flat MPC, hierarchical admission/budget LP, the two-level routing LP, and the offline oracle bound) are solved with HiGHS via `scipy.optimize.linprog/milp`. Orbital mechanics, contact windows, and link feasibility come from Hypatia’s satellite-generation and topology modules (`hypatia/satgenpy`), vendored unmodified.

4.6 Quality-table provenance

The utility function’s quality term s_q (§6, §3) is grounded in measured downstream FLAIR-1 segmentation mIoU, source label `flair-unet-r34-rgbie / val17050 / 2026-08-31` (full 7,050-image validation population; see §5 for the measurement itself). All results in §6 and §7 use this real, measured table (`hypatia_sim/oec_sim/quality_table.json`), not the placeholder fallback (`config.QUALITY_TABLE_FALLBACK`) that the code falls back to when this file is absent.

5 Compression Model Results on FLAIR-1

This section reports the on-board RQ-VAE compression model’s reconstruction quality, the downstream segmentation task performance it induces (which grounds the utility function’s quality term s_q), and on-board compute profiling.

5.1 Reconstruction quality: dedicated models

Table 3 reports reconstruction quality for dedicated RQ-VAE models trained at each depth, evaluated on the full 7,050-image FLAIR-1 validation split, fixed 8×8 spatial latent grid. Depths 2, 3, and 4 have no dedicated trained model (training configs exist but were not run); §5.2 covers how those conditions are obtained instead.

Table 3: Dedicated FLAIR-1 RQ-VAE models, full validation split.

Model	Payload	PSNR (dB)	SSIM	LPIPS	FID	Compression
$8 \times 8 \times 1$	88 B	20.63	0.4560	0.4595	71.33	8,937:1
$8 \times 8 \times 8$	704 B	21.02	0.4643	0.4779	73.19	1,117:1
$8 \times 8 \times 16$	1,408 B	21.06	0.4899	0.5039	125.78	559:1

5.2 The truncation experiment: is reuse viable?

To avoid training a dedicated model per depth, we tested whether a single depth-16 model, decoded using only its first N of 16 residual stages at inference (`forward_partial_code`), can substitute for a dedicated depth- N model. Table 4 reports this truncated-inference condition against the same depth-16 checkpoint.

Table 4: Truncated inference from one depth-16 checkpoint.

Truncated to	PSNR (dB)	SSIM	LPIPS	FID
1 stage	18.67	0.4793	0.5948	225.25
2 stages	19.72	0.4838	0.5570	197.76
4 stages	20.35	0.4831	0.5314	163.90
8 stages	20.73	0.4861	0.5171	143.19
16 stages	21.06	0.4899	0.5039	125.78

Result: reuse is not viable. At depth 1, truncated inference is 2dB worse in PSNR and over $3\times$ worse in FID than the dedicated depth-1 model (225.25 vs. 71.33). Even at depth 8, where the gap narrows, truncated inference remains measurably behind dedicated training (FID 143.19 vs. 73.19). Consequently, the depth-2/4/8 downstream conditions reported in §5.3 use truncated inference from the depth-16 checkpoint and should be read as a *lower bound* on what a dedicated model at that depth would achieve, not its true ceiling.

5.3 Downstream task performance

Pixel-fidelity metrics (PSNR, LPIPS) do not measure whether a reconstruction is still useful for the mission task. We evaluate each compression depth’s reconstructions on FLAIR-1 semantic segmentation (15 land-cover classes) using IGNF’s pretrained `FLAIR-INC_rgbie_15cl_resnet34-unet` model, scored by mean IoU (mIoU) over the full 7,050-image validation population (Table 5).

Table 5: Downstream segmentation mIoU by compression depth. s_q is floor-anchored: $s_q = (\text{mIoU}_q - \text{mIoU}_{\text{floor}}) / (\text{mIoU}_{\text{ref}} - \text{mIoU}_{\text{floor}})$.

Condition	mIoU	s_q (floor-anchored)
blank (NIR + elevation only, floor)	6.08%	—
$q = 1$ (truncated)	12.60%	0.104
$q = 2$ (truncated)	17.53%	0.182
$q = 4$ (truncated)	22.54%	0.262
$q = 8$ (truncated)	25.68%	0.312
$q = 16$ (dedicated)	28.70%	0.360
uncompressed (ref)	68.87%	—

This measured spread (s_q : +247% from $q = 1$ to $q = 16$) is far wider than the pixel-fidelity proxy it replaces ($1 - \text{LPIPS}$: only +12% over the same range), and is the reason the unified utility function’s quality term is grounded in mIoU rather than pixel fidelity (§3).

Per-class results show this recovery is uneven: fine-texture classes (swimming pools, coniferous/deciduous forest) lose 70–99% of their achievable IoU at $q = 1$, while broad classes (agricultural land, greenhouses) lose only 28–35%. Per-class detail is reported as supplementary analysis; the utility function uses overall mIoU by design.

[CONFIRM WITH XUANHAO: The uncompressed-reference mIoU (68.87%) is higher than both the checkpoint’s expected reproduction target ($\approx 54.43\%$) and its Hugging Face model-card self-reported value (58.6%). A domain-leakage check between the validation population and the training domain came back clean, and per-class scores are structurally sane, but this has not

been verified against FLAIR-1’s true held-out test split. Treat 68.87% as internally consistent for comparing depths against each other, not as a verified match to an external published number.]

5.4 On-board compute profiling

Encode time was measured on an NVIDIA A6000 at $8 \times 8 \times 1$ and $8 \times 8 \times 8$: $12.34 \text{ ms} \pm 0.03\text{--}0.05 \text{ ms}$ per image at both depths, i.e. encode time is depth-independent (dominated by the encoder CNN forward pass, not the residual-quantization loop). Per-image energy at 30 W (Jetson AGX Orin mid-band assumption, since the A6000 used for timing is not a flight part): encode = 0.370 J ; downlink at $q = 16$, $11,264 \text{ bits}$ at $2 \times 10^{-7} \text{ J/bit} = 2.25 \times 10^{-3} \text{ J}$. Encode dominates downlink energy by $\sim 164 \times$ per image ($\sim 230 \times$ over a run’s full depth mix), and since it is depth-independent, a Joule-denominated cost term cannot drive depth choice — depth selection in this system is a *bandwidth* decision, not an energy one (§3 discusses how the utility’s cost term is scaled accordingly).

6 Unified Utility: Baselines, Method, and Variants

6.1 Legacy vs. unified utility: not the same scale

Two utility functions are implemented, selected by `--utility legacy|unified`. **Legacy** is a single weighted quality-times-freshness sum with no tardiness, cost, or fairness term, using a pixel-fidelity quality proxy ($1 - \text{LPIPS}$). **Unified** adds a tardiness penalty, a resource-cost penalty, and a maximin fairness floor, and replaces the proxy with the measured downstream mIoU term s_q from §5.3 (full formulation in §3). These two modes are *not* on the same scale and must not be compared directly in a single table; Table 6 reports each mode’s own best-fixed-depth-greedy vs. MPC margin, from the `gbs-limited` scenario, to show how switching the quality term changes the conclusion.

Table 6: MPC vs. best fixed-depth greedy, gbs-limited scenario, seed 42.

Utility mode	mpc	best fixed greedy	MPC margin
legacy ($1 - \text{LPIPS}$ quality)	64.95	64.15 (fixed-8)	+1.2%
unified (real mIoU quality)	18.13	13.71 (fixed-8)	+32.2%

Under the pixel-fidelity proxy, MPC essentially ties the best fixed-depth greedy baseline. Once quality reflects the real downstream task, MPC’s percentage margin is $\approx 26 \times$ larger. This is the central result motivating the unified utility: the legacy proxy under-states MPC’s value because it is nearly flat across compression depths (§5.3).

6.2 Compared algorithms

- **greedy-fixed- $\{2,4,8,16\}$** : always encode at a fixed depth, transmit whatever fits in the current slot’s remaining capacity.
- **greedy-adaptive**: greedy, but adapts encode depth to queue length.
- **mpc**: flat rolling-horizon MILP (§3), static shortest-path routing.
- **mpc-congestion**: flat MPC with predictive, congestion-aware routing (M/M/1-style edge cost, iterated to a fixed point).
- **mpc-hier**[**CONFIRM WITH XUANHAO: scope of this variant to keep/drop is pending Xuanhao’s confirmation, see note below**]: two-level hierarchy — a slow admission/budget LP plus a fast per-task depth MILP, with explicit task drop/abandon logic.

- **mpc-hier-route**: **mpc-hier** plus a routing MPC that freezes a concrete path per 10-minute macro-epoch.
- **mpc-2level**: a routing MPC (multicommodity-flow LP over candidate paths) and the depth MPC iterated to a fixed point as *peers*, not hierarchically.

[CONFIRM WITH XUANHAO: Xuanhao’s email asked to drop “the previous **mpc-hier** results where the two MPCs are both used for depth selection.” **FORMULATION.md** describes **mpc-hier**’s two levels as: upper = LP over a *continuous relaxed depth* $x_{k,q} \in [0, 1]$ (for budgeting), lower = MILP over *integral depth* $x_{k,q}$ – i.e. both levels of **mpc-hier** specifically operate on depth, with static routing at both. This is different from **mpc-hier-route/mpc-2level**, which are an explicit routing-MPC + depth-MPC split. This textual match makes **mpc-hier** the likely candidate for what Xuanhao means, but all schedulers are reported below pending his direct confirmation – see §3.9 for the full two-level description.]

6.3 Ten schedulers, one run

Table 7 reports all ten schedulers on the **fabric-limited** scenario (where routing choices are measurable), unified utility, real mIoU quality table, single run/seed (42). Every row is from the same run, so utility, delivery, on-time rate, and bound-gap are directly comparable across schedulers.

Table 7: Scheduler comparison, fabric-limited scenario, seed 42. Bound (offline HiGHS upper bound, §3) = 18.99.

Scheduler	Utility	Delivery%	On-time%	Gap to bound
greedy-fixed-2	6.43	100%	88.7%	66.1%
greedy-fixed-4	11.08	100%	88.7%	41.7%
greedy-fixed-8	13.71	100%	88.6%	27.8%
greedy-fixed-16	8.80	100%	81.5%	53.7%
greedy-adaptive	8.80	100%	81.5%	53.7%
mpc	18.76	87.9%	86.3%	1.2%
mpc-congestion	18.86	89.3%	87.6%	0.7%
mpc-hier	14.76	78.7%	75.0% (39 dropped)	22.3%
mpc-hier-route	14.78	81.9%	78.6% (40 dropped)	22.2%
mpc-2level	18.90	90.3%	88.6%	0.5%

Note: **greedy-adaptive**’s row is numerically identical to **greedy-fixed-16** in this run — in this scenario, the adaptive policy’s depth choice converges to always selecting depth 16, so this is a genuine coincidence in the underlying decisions (verified in §7), not a duplicated row.

6.4 Individual utility components

Table 8 decomposes total utility into its four additive terms, from the same run as Table 7 (*hypatia_sim/oec_scenario_couplings/summary.txt*). U = quality – tardy – cost + fair; $u_{\min}$ is the optimized maximin fairness floor and Jain’s index is reported as a diagnostic only (not MILP-representable, so not itself optimized; see §3).

What drives the gap: admission control, not routing. **mpc-2level** wins outright, but **mpc-congestion** (predictive routing only, no full flow-optimization) sits within 0.04 utility of it. A five-point ISL-rate sweep (§8) found the dedicated routing-LP’s own marginal contribution tops out at +0.84% utility and is exactly zero above 1.5 Mbps ISL. The real, structural ≈ 5 -point gap is between the no-admission-control family (**mpc**, **mpc-congestion**, **mpc-2level**, ≈ 18.8 – 18.9) and the admission-control family (**mpc-hier**, **mpc-hier-route**, ≈ 14.8) — whether to drop a task early matters far more than how cleverly the kept tasks are routed.

Table 8: Utility component decomposition, fabric-limited scenario, seed 42.

Scheduler	Total	Quality	−Tardy	−Cost	+Fair	$u_{\min}$	Jain	Enc. (kJ)
mpc	18.757	16.344	−0.029	−2.121	4.564	0.7131	0.9978	4170.2
greedy-adaptive	8.797	16.170	−1.502	−2.350	−3.521	−0.5502	0.9118	4742.3
greedy-fixed-2	6.431	8.414	−0.813	−1.322	0.152	0.0237	0.9331	4742.3
greedy-fixed-4	11.076	12.089	−0.813	−1.469	1.269	0.1983	0.9670	4742.3
greedy-fixed-8	13.711	14.394	−0.814	−1.763	1.894	0.2959	0.9748	4742.3
greedy-fixed-16	8.797	16.170	−1.502	−2.350	−3.521	−0.5502	0.9118	4742.3
mpc-congestion	18.864	16.447	−0.028	−2.151	4.597	0.7182	0.9985	4235.1
mpc-hier	14.761	13.743	−0.085	−1.536	2.639	0.4124	0.9846	3733.3
mpc-2level	18.898	16.503	−0.030	−2.173	4.597	0.7183	0.9987	4281.9
mpc-hier-route	14.779	13.811	−0.055	−1.561	2.584	0.4038	0.9848	3885.9

Route-freezing does not survive a LEO fabric. A route frozen for a 10-minute macro-epoch (**mpc-hier-route**) is usable at its actually-executed slot only 7.7% of the time, because ground-station contact windows (227–265s, Table 1) are shorter than any freeze period worth having. This is a structural property of the constellation geometry, not a tuning failure.

6.5 Solve-time and energy efficiency

mpc-hier/mpc-hier-route solve 3–4× faster per call (mean 43.5–43.6 ms) than the flat/two-level MPCs (123.9–168.5 ms mean, up to 3,088 ms max for **mpc-2level**), at the cost of the utility gap above. **mpc-2level** solves 365 times over the run (vs. 113–121 for the other MPC variants), for 45.2 s total solve time vs. 20.4 s for flat **mpc** — recall from §4 that these are individual-solve statistics summed, not simulated wall-clock time.

7 Decision Distributions

This section reports the compression-depth chosen per task, per scheduler, from the same fabric-limited run (seed 42) as Tables 7–8, so utility, delivery, and decision-distribution results are directly comparable. Distributions were computed by aggregating the `depth` column of each `task_outcomes_*.csv` (64 tasks per scheduler); Table 9 reports the count of tasks assigned each depth, plus tasks dropped or rejected before a depth was ever assigned.

Table 9: Depth-selection distribution per scheduler (of 64 tasks), fabric-limited scenario, seed 42.

Scheduler	$D=2$	$D=4$	$D=8$	$D=16$	Dropped/rejected
greedy-fixed-2	64	0	0	0	0
greedy-fixed-4	0	64	0	0	0
greedy-fixed-8	0	0	64	0	0
greedy-fixed-16	0	0	0	64	0
greedy-adaptive	0	0	0	64	0
mpc	0	0	2	62	0
mpc-congestion	0	0	1	63	0
mpc-hier	1	11	35	17	39
mpc-hier-route	1	12	37	14	40
mpc-2level	0	0	0	64	0

Fixed-depth greedy baselines behave exactly as specified (sanity check: 64/64 tasks at the configured depth for every **greedy-fixed-*** row). **greedy-adaptive converges to depth 16**

for every task in this run — confirming numerically why its row is identical to **greedy-fixed-16** in Tables 7–8: under this scenario’s contention level, the adaptive policy’s queue-length trigger never fires away from the maximum depth. **The no-admission-control MPC family (mpc, mpc-congestion, mpc-2level) is overwhelmingly depth-16**, using a lower depth only rarely (mpc: 2/64; mpc-congestion: 1/64) and never dropping a task. **The admission-control family (mpc-hier, mpc-hier-route) shows real depth diversity** across all four levels, but this comes with substantial admission/drop activity (39–40 of 64 tasks dropped or rejected before delivery) — the mechanism described in §3.9 trading deliverability for smaller, faster-solved problems. This is the same finding stated at the utility level in §6 (§3 §3.9): admission control, not depth or routing sophistication, is what distinguishes the two scheduler families.

[PENDING RERUN: This table is reported for one scenario (fabric-limited) and one seed. Repeating it for the gbs-limited scenario, and for the satellite-/GBS-count sweep proposed in §8, would show whether the depth-diversity pattern in the admission-control family is scenario-specific or general.]

8 Sensitivity to Link Rate, Constellation, and Ground-Segment Size

8.1 GBS aggregate rate sweep: needs to be rerun

`oec_sim/sweep.py` committed a grid of 6 GBS aggregate rates $\times$ 5 seeds $\times$ 6 schedulers (180 rows, `hypatia_sim/oec_scenario/sweep/results.csv`). [PENDING RERUN: This sweep was generated in the same commit that unified the utility function, but *before* the real downstream mIoU quality table was wired in one commit later – its own commit message states the quality table was “not yet run on the server ... provisional until then” at that point, and the file has not been regenerated since. Its utility values (≈ 57 –64) are on the placeholder-quality scale, not the real-quality scale (≈ 6 –19) used everywhere else in this paper. **This sweep must be rerun against the current quality_table.json before its exact numbers are reported as final** – flagging this rather than transcribing stale numbers.] The qualitative shape of the earlier (placeholder-quality) run is: the best *fixed* depth climbs $2 \rightarrow 4 \rightarrow 8 \rightarrow 16$ as the GBS rate rises, and **mpc** beat the best fixed depth in 5 of 6 rate regimes, tying only at the highest rate tested (no scarcity, “always max depth” already near-optimal) – worth confirming still holds once rerun with real quality.

8.2 ISL rate sweep

A parallel 5-point ISL-rate sweep (0.5–2.0 Mbps, 3 seeds, `results_routing.csv`) isolates the routing MPC’s own marginal contribution: at most +0.84% utility, and exactly 0% above 1.5 Mbps ISL (converges to flat MPC once there is nothing to route around). This sweep has the identical staleness issue as §5.1 (placeholder quality table) and needs the same rerun before its exact numbers are final.

8.3 Proposed: satellite-count and GBS-count sweeps

Both axes are existing configuration knobs, not new code: `config.CONSTELLATIONS` already defines four named presets varying satellite count (`kuiper-630`: 1,156 sats; `starlink-550`: 1,584 sats; `telesat-1015`: 351 sats; `small-walker`: 6 sats, a topology-validation toy scenario not suited for a results sweep), and `config.GROUND_STATIONS` is a plain list whose length sets $|G|$. A satellite-count sweep can therefore run the existing harness (`sweep.py`) across the three real constellation presets (or intermediate synthetic (planes,sats/plane) tuples added to `CONSTELLATIONS`); a GBS-count sweep can add/remove entries from `GROUND_STATIONS` (e.g. a 2-, 4-, 8-station subset of Hypatia’s top-100 city list) and rerun. [PENDING RERUN: Neither sweep has been run yet – this is a proposed design, not a result. Both should use the current real quality table from the outset to avoid the staleness problem in §5.1–5.2.]

9 A Learned Scheduling Policy: PPO vs. MPC

Every scheduler in §6 solves an optimization problem from scratch at every re-plan. This section evaluates the alternative: train a policy offline so that, at run time, it only has to evaluate a small neural network rather than solve a fresh MILP.

[CONFIRM WITH XUANHAO: These results are on **legacy utility scale** (§3), not unified — the trained policy (`ppo.oec.zip`) optimized the old single-factor reward and has not been retrained under `--utility unified`. Do not compare the utility numbers in this section against Tables 7–8, which are on the unified scale; retraining under the unified reward is future work.]

9.1 Setup

The environment (`oec.sim/rl.env.py`) is a Gymnasium environment with a `Discrete(6)` action space — commit one of 4 depths, defer, or drop a task — decided per active task rather than per slot, so the action space size does not depend on how many tasks are active or how large the constellation is. The reward matches the MILP objective plus a potential-based backlog-shaping term. The policy was trained with PPO (Proximal Policy Optimization) for 300k timesteps across load regimes 1.5–2.5 Mbps and 10 seeds (~ 4 minutes on a CPU), then evaluated both in-distribution and out-of-distribution (a load regime never seen in training).

9.2 Results

Table 10 reports utility, delivery rate, and per-decision compute time for two representative seeds per regime.

Table 10: RL (PPO) vs. MPC, legacy utility scale, two seeds per regime.

Regime	RL utility	RL delivered%	RL μ s/decision	MPC utility	MPC μ s/decision
in-dist (2.0 Mbps)	58.13 / 52.31	100%	~ 500 –1,400	61.58 / 55.55	$\sim 145,000$ –243,000
OOD-light (4.0 Mbps)	58.13 / 52.31	100%	~ 500 –1,400	62.44 / 56.14	$\sim 145,000$ –243,000
OOD-heavy (0.75 Mbps)	54.76 / 50.83	100%	~ 500 –700	58.92 / 53.27	$\sim 163,000$ –173,000

Three findings, two anticipated and one not:

1. **PPO loses to MPC on utility, as expected** — 5–8% below MPC in every regime tested. Solving the optimization problem freshly at every decision still finds a better plan than pattern-matching against training experience.
2. **PPO is ~ 250 – $400\times$ faster per decision** (microseconds vs. hundreds of milliseconds) — the anticipated inference-cost advantage of a learned policy holds up strongly, and is the practical reason a learned policy might still be attractive on compute- and power-constrained flight hardware despite the utility gap.
3. **Unexpected: the policy’s actions are provably rate-independent.** Its observation vector includes task properties, active-task count, and mean freshness, but no capacity or congestion feature — so its decisions are bit-for-bit identical between the 2.0 Mbps and 4.0 Mbps regimes at a fixed seed (visible directly in Table 10). It still delivers 100% of images even at the most constrained regime tested (0.75 Mbps, where **greedy-fixed-8** collapses to a fraction of its unconstrained utility) purely because it happened to learn conservative depth choices, not because it senses congestion. This is *accidental* robustness, not adaptive robustness — the clear next step is adding a residual-bandwidth/congestion feature to the observation, which the original environment design called for but the shipped version omitted to keep the observation vector smaller.

Two comparison-fairness issues in the RL evaluation path were identified and fixed before these numbers were produced: the environment had recorded RL deliveries with no propagation delay while every other scheduler charged the real route delay (effect measured at 0.0017% utility — negligible relative to the 30-second slot and 300-second freshness constant, a correctness fix rather than a result change), and the RL scheduler’s own utility accumulation bypassed the shared `utility.run_utility` path and would have silently omitted the fairness term under `--utility unified`. Both now go through the same accounting as every other scheduler in this paper.

10 Conclusion

This paper studied on-board compression depth and downlink scheduling for Orbital Edge Computing as one coupled decision problem, and found that the choice of *quality signal* used to reward that decision matters more than any single scheduler design choice tested. Grounding utility in measured downstream segmentation performance rather than pixel fidelity changed MPC’s measured advantage over the best fixed-depth baseline from a marginal 1.2% to a substantial 32.2%, because pixel fidelity is nearly flat across the compression-depth range studied here while task-relevant quality is not (+12% vs. +247%). A scheduler is only as good as the signal it is scored against.

With that signal in place, a single unified utility function reconciled five previously disagreeing scoring definitions, and a ten-scheduler comparison under a deliberately routing-sensitive capacity regime showed that the strongest variant (`mpc-2level`, a routing MPC and a depth MPC solved as peers) closes to within 0.5% of the offline optimality bound. The more consequential finding, however, was architectural rather than numerical: the ≈ 5 -point utility gap that separates the strongest and weakest MPC variants tracks *admission control* (whether a scheduler explicitly drops an overwhelmed task rather than let it rot in queue), not routing sophistication — the dedicated routing optimizer’s own marginal contribution tops out at +0.84% utility and disappears entirely once link rates are not scarce. A related structural result is that freezing a concrete route for longer than a few minutes does not survive this constellation’s short ground-contact windows, regardless of how that freeze period is tuned. Finally, a learned (PPO) policy traded a consistent 5–8% utility gap for a 250–400 $\times$ reduction in per-decision compute time, with an unanticipated caveat: without a congestion feature in its observation, its apparent robustness to link scarcity is accidental rather than adaptive.

What remains open. Several results in this paper are explicitly flagged, not silently finalized: the uncompressed-reference mIoU used to anchor the quality signal has not been verified against FLAIR-1’s official held-out test split (§5.3); the scope of `mpc-hier` relative to Xuanhao’s request to drop certain prior results is pending his direct confirmation (§6); and both link-rate sensitivity sweeps in §8 were generated against a placeholder quality table and need to be rerun against the real, measured one before their exact numbers are reported as final. The proposed satellite-count and ground-station-count sweeps (§8) and a retraining of the PPO baseline under the unified reward (§9) are natural next steps that require no new code, only additional runs. None of these open items affect the paper’s central conclusion — that a scheduler’s measured value depends first on what it is being asked to optimize, and only second on how cleverly it optimizes it — but resolving them would sharpen every number built on top of that conclusion.

References

- [1] Simon Kassing, Debopam Bhattacharjee, André Baptista Águas, Jens Eirik Saethre, and Ankit Singla. Exploring the “internet from space” with hypatia. In *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.

- [2] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [3] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2019.
- [4] Anatol Garioud, Stéphane Peillet, Eva Bookjans, Sébastien Giordano, and Boris Wattrelos. Flair: French land cover from aerospace imagery. Institut national de l’information géographique et forestière (IGNF), 2022.